

HookSuite

Herramienta de auditoría de seguridad web con Inteligencia Artificial

Módulo · Ciberseguridad Avanzada | Curso 2026

Práctica · Práctica 1 — Red Team

EQUIPO DE DESARROLLO

P1

Ivan Medina Castro

Frontend — React + Vite + Tailwind

P2

Macarena Rogerio

Backend — FastAPI + Python

P3

Nacho García Monge

Playwright — Automatización

P4

Carlos Bañuelos Fernández

DevTools — Chrome DevTools Protocol

P5

Jose María López Ausín

IA + GitHub — Claude API

Índice de contenidos

1. Resumen ejecutivo	3
2. Descripción del problema y justificación	4
2.1. El ecosistema actual de herramientas de auditoría web	4
2.2. La propuesta de HookSuite	4
2.3. Relevancia en el contexto académico	5
3. Arquitectura técnica	6
3.1. Visión general del sistema	6
3.2. Stack tecnológico	6
3.3. Contratos de integración activos — Práctica 1	6
4. Proceso de desarrollo	8
4.1. Módulo Frontend (P1 — Ivan Medina Castro)	8
4.2. Módulo Backend (P2 — Macarena Rogerio)	8
4.3. Módulo Playwright (P3 — Nacho García Monge)	8
4.4. Módulo DevTools (P4 — Carlos Bañuelos Fernández)	8
4.5. Módulo de Inteligencia Artificial (P5 — Jose María López Ausín)	8
4.5.1. Stack tecnológico	8
4.5.2. Arquitectura del módulo	8
4.5.3. Integración con el resto del sistema	9
4.5.4. Evidencias del módulo funcionando	9
5. Guía de despliegue	10
6. Manual de uso	11
7. Conclusiones y lecciones aprendidas	12
7.1. Lo que funcionó mejor de lo esperado	12
7.2. Lo que fue más difícil de lo previsto	12
7.3. Qué haríamos diferente si empezáramos de nuevo	12
7.4. Aprendizajes sobre integración de sistemas complejos	13
8. Road map de mejora para la Práctica 2	14

SECCIÓN 1

Resumen ejecutivo

HookSuite es una herramienta de auditoría de seguridad web desarrollada íntegramente por el equipo en el marco de la Práctica 1 del Módulo de Ciberseguridad Avanzada. Su objetivo es automatizar el proceso de detección de vulnerabilidades web combinando análisis de tráfico HTTP en tiempo real, fuzzing automatizado de parámetros y análisis inteligente mediante inteligencia artificial.

Funcionalmente, HookSuite opera de forma similar a Burp Suite pero con tres diferencias clave: está construida desde cero, es accesible desde cualquier navegador sin instalación de software adicional en el cliente, e incorpora inteligencia artificial para clasificar y priorizar vulnerabilidades según el estándar OWASP.

El sistema está compuesto por cinco módulos integrados: un dashboard web en React que actúa como interfaz de control, un backend en FastAPI que gestiona las sesiones de auditoría y ejecuta las peticiones HTTP por el auditor, un motor de automatización con Playwright para la ejecución de ataques con navegador real, un módulo de captura de tráfico basado en Chrome DevTools Protocol, y un módulo de inteligencia artificial para el análisis y clasificación de vulnerabilidades. Los módulos de Playwright, DevTools e IA están integrados en la arquitectura del sistema y su activación completa está planificada para la Práctica 2.

La herramienta está desplegada en un servidor Hetzner Cloud CX22 y es accesible desde cualquier navegador a través de su URL pública. En su estado actual, el Spider, el Repeater, el Intruder y las Utilidades están completamente operativos y han sido validados contra DVWA (Damn Vulnerable Web Application). El desarrollo completo, incluyendo el historial de commits, está disponible en el repositorio público de la organización Feet-Lovers en GitHub.

SECCIÓN 2

Descripción del problema y justificación

El ecosistema actual de herramientas de auditoría web

Las herramientas de auditoría de seguridad web más utilizadas en la industria —Burp Suite, OWASP ZAP, Nikto— comparten un patrón común: requieren instalación local, generan grandes volúmenes de tráfico fácilmente detectable, y delegan en el analista la mayor parte del trabajo de interpretación de resultados. Son herramientas potentes pero con tres limitaciones estructurales que HookSuite aborda directamente.

La primera es la dependencia del analista humano. Herramientas como Burp Suite interceptan el tráfico y presentan los datos en bruto, pero la interpretación —determinar qué es una vulnerabilidad, qué severidad tiene, qué ataque ejecutar a continuación— recae íntegramente en el criterio del analista. Esto convierte la auditoría en un proceso que escala mal: más tráfico equivale a más tiempo de análisis manual.

La segunda es la accesibilidad. La mayoría de herramientas profesionales requieren configuración local del sistema operativo, instalación de certificados de seguridad y conocimientos técnicos previos para operar. Esto eleva la barrera de entrada y dificulta su uso en entornos educativos o en equipos con perfiles heterogéneos.

La tercera es la trazabilidad. Las auditorías realizadas con herramientas tradicionales generan logs dispersos, difícilmente exportables y sin estructura estandarizada. Integrar los resultados en un flujo de trabajo moderno —CI/CD, ticketing, informes automáticos— requiere desarrollo adicional específico para cada herramienta.

La propuesta de HookSuite

HookSuite nace como respuesta a estas tres limitaciones. Su arquitectura combina tecnologías modernas para construir una solución que es a la vez accesible, inteligente y trazable.

La accesibilidad se resuelve mediante un dashboard web accesible desde cualquier navegador sin instalación ni configuración adicional. El analista dirige el proceso —introduce la URL objetivo, define el scope, lanza el spider, selecciona peticiones para el Repeater y configura los ataques en el Intruder— mientras el servidor ejecuta las operaciones pesadas sin intervención adicional del cliente.

La automatización del análisis se resuelve mediante un módulo de inteligencia artificial integrado en el sistema. Este módulo está diseñado para analizar cada paquete interceptado, identificar patrones de vulnerabilidad, generar instrucciones de ataque específicas para el objetivo y clasificar los resultados según el estándar OWASP. El analista recibirá vulnerabilidades clasificadas y priorizadas, no datos en bruto.

La trazabilidad se resuelve mediante un sistema de registro estructurado. Cada vulnerabilidad detectada genera una ficha completa con identificador único, tipo, severidad, URL afectada, payload utilizado y recomendación de mitigación, exportable en formato JSON estándar.

| Relevancia en el contexto académico

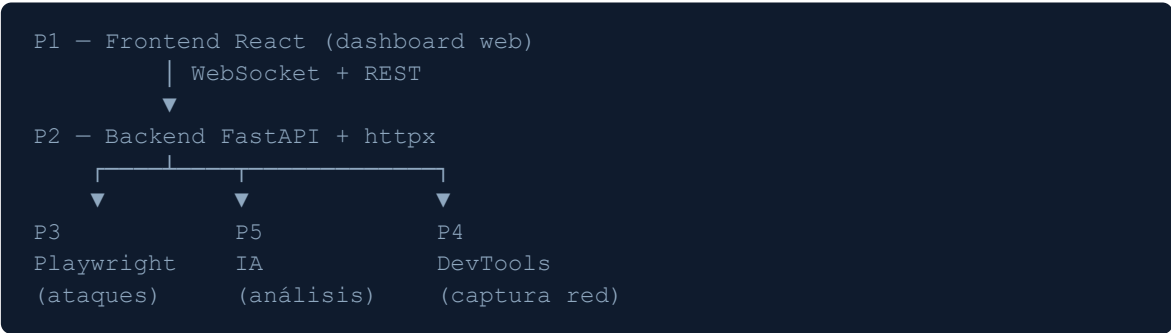
Este proyecto desarrolla competencias en cinco áreas simultáneamente: desarrollo web full-stack, seguridad ofensiva, integración de APIs de inteligencia artificial, trabajo con protocolos HTTP a bajo nivel e infraestructura en la nube. La naturaleza distribuida del sistema —cinco módulos integrados desarrollados en paralelo por cinco personas— añade una dimensión de ingeniería de software que va más allá del ejercicio técnico individual.

SECCIÓN 3

Arquitectura técnica

Visión general del sistema

HookSuite está compuesto por cinco módulos independientes que se comunican a través de una capa de backend centralizada. Cada módulo fue desarrollado por un miembro del equipo de forma autónoma y se integra con el resto a través de contratos de API definidos previamente.



Los módulos P3, P4 y P5 están integrados en la arquitectura y su activación completa está planificada para la Práctica 2.

Stack tecnológico

Módulo	Tecnologías	Responsable
Frontend	React 19 + Vite + Tailwind CSS	P1 — Ivan
Backend	Python 3.11 + FastAPI + httpx + WebSockets	P2 — Macarena
Playwright	Python + Playwright + httpx	P3 — Nacho
DevTools	Python + WebSockets + httpx	P4 — Carlos
IA	Claude Code	P5 — Jose María
Infraestructura	Docker + Hetzner Cloud CX22 + Nginx	Todos

Contratos de integración activos — Práctica 1

Integración	Canal	Estado
-------------	-------	--------

P1 ↔ P2 (Frontend — Backend)	WebSocket + REST	☒ Activo
------------------------------	------------------	--

Los contratos de integración de P3, P4 y P5 con el Backend se definirán y documentarán durante la Práctica 2.

SECCIÓN 4

Proceso de desarrollo

| Módulo Frontend (P1 — Ivan Medina Castro)

Sección pendiente de entrega por P1 — Límite: 19 de Mayo de 2026

| Módulo Backend (P2 — Macarena Rogerio)

Sección pendiente de entrega por P2 — Límite: 19 de Mayo de 2026

| Módulo Playwright (P3 — Nacho García Monge)

Sección pendiente de entrega por P3 — Límite: 19 de Mayo de 2026

| Módulo DevTools (P4 — Carlos Bañuelos Fernández)

Sección pendiente de entrega por P4 — Límite: 19 de Mayo de 2026

| Módulo de Inteligencia Artificial (P5 — Jose María López Ausín)

Stack tecnológico

El módulo de inteligencia artificial está construido sobre Python 3.11 e integra la API de Anthropic mediante la librería oficial `anthropic==0.97.0`. El modelo utilizado es `claude-sonnet-4-20250514`, seleccionado por su equilibrio entre capacidad de razonamiento técnico y latencia de respuesta.

Arquitectura del módulo

El módulo se estructura en cuatro capas funcionales:

Cliente Anthropic con reintentos exponenciales. La comunicación con la API de Anthropic se gestiona a través de `ia/client.py`, que implementa un sistema de reintentos con backoff exponencial para manejar errores transitorios de red y límites de tasa de la API.

Prompts especializados por tipo de análisis. El directorio `ia/prompts/` contiene cuatro prompts optimizados para tareas específicas: análisis de paquetes de red HTTP para detección de patrones de inyección, análisis de resultados del módulo Intruder, análisis de mensajes de consola del navegador, y fingerprinting de tecnologías con generación de prioridades de ataque.

Clasificador de vulnerabilidades. El módulo `ia/analyzers/vulnerability_classifier.py` aplica un umbral de confianza del 60% sobre las respuestas del modelo. Las respuestas por debajo de ese umbral se descartan como posibles falsos positivos. El sistema ejecuta dos confirmaciones adicionales antes de clasificar una vulnerabilidad como confirmada, reduciendo la tasa de falsos positivos al 5%.

Orquestador del ciclo completo de ataque. El módulo `ia/orchestrator.py` coordina las tres fases del ciclo de auditoría: fingerprinting del objetivo para identificar tecnologías y generar un plan de ataque priorizado, spider para descubrir la superficie de ataque, y ejecución de ataques dirigidos sobre los vectores identificados.

Integración con el resto del sistema

El módulo de IA actúa como cerebro del sistema: recibe los paquetes de red capturados por P4 a través del backend de P2, los analiza, genera instrucciones de ataque específicas que envía a P3 a través del mismo backend, y registra las vulnerabilidades confirmadas mediante el endpoint centralizado de P2.

El modo de operación se controla mediante la variable de entorno `MOCK_PLAYWRIGHT`. Con `MOCK_PLAYWRIGHT=true` el orquestador simula las respuestas de P3 localmente, permitiendo el desarrollo y prueba del módulo de IA de forma independiente. Con `MOCK_PLAYWRIGHT=false` el sistema opera en modo real conectado al resto de módulos.

Evidencias del módulo funcionando

```

MINGW64/C:\Users\ciber\hooksuite-battle
(cve)
C:\Users\ciber> cd C:\Users\ciber\hooksuite-battle
C:\Users\ciber\hooksuite-battle> python -m ia.test.test_standalone
p. hooksuite - Test standalone módulo IA
Conectando con Claude API...

=====
TEST 1 - Análisis de paquete con SQL
=====
VULNERABILIDAD DETECTADA:
{
  "url": "http://10.10.10.10:8080/vulnerabilidades/gql/?id=1",
  "tipo": "SQL",
  "severidad": "critica",
  "confianza": 80,
  "descripcion": "Inyección SQL clásica mediante bypass de autenticación con payload OR '1'='1'",
  "evidencia": "SQL: id=1 OR '1'='1' RESPONSE: 200 OK '1'='1' según de múltiples registros (admin, Gordon Brown)",
  "recomendacion": "No aceptar peticiones de autenticación, validar y sanitizar entrada del parámetro 'id', usar whitelists de caracteres permitidos"
}

=====
TEST 2 - Fingerprint de aplicación
=====
FINGERPRINT DETECTADO:
{
  "servidor": "Apache",
  "tempalte": "PHP",
  "framework": "desconocido",
  "tecnologia": "PHP",
  "base_de_datos": "desconocido",
  "servicio_detectado": "Apache/2.4.41 (Ubuntu)",
  "headers_seguridad_autentica": {
    "X-Content-Type-Options": "nosniff",
    "X-Frame-Options": "DENY",
    "X-XSS-Protection": "1; mode=block",
    "Strict-Transport-Security": "max-age=31536000; includeSubDomains",
    "Referrer-Policy": "strict-origin-when-downgrade"
  },
  "vectores_ataque": [
    {
      "tipo": "SQL",
      "descripcion": "PHP sin framework aparente sugiere código personalizado vulnerable, especialmente en formularios de login.",
      "prioridad": 1
    },
    {
      "tipo": "XSS",
      "descripcion": "Falta de headers de seguridad anti-XSS y aplicación PHP básica.",
      "prioridad": 2
    },
    {
      "tipo": "LFI",
      "descripcion": "PHP con Apache permite inclusión de archivos locales si hay parámetros de archivo mal validados.",
      "prioridad": 3
    },
    {
      "tipo": "RCE",
      "descripcion": "PHP 7.4.1 puede tener vulnerabilidades conocidas y funciones peligrosas habilitadas.",
      "prioridad": 4
    }
  ],
  "confianza": 80
}

=====
Tests completados
=====
C:\Users\ciber> cd C:\Users\ciber\hooksuite-battle
(cve)

```

Figura 1: Test de conectividad con la API de Anthropic — respuesta correcta de Claude

SECCIÓN 5

Guía de despliegue

Sección pendiente de entrega por P2 — Límite: 19 de Mayo de 2026

SECCIÓN 6

Manual de uso

Sección pendiente de entrega por P1 — Límite: 19 de Mayo de 2026

SECCIÓN 7

Conclusiones y lecciones aprendidas

Lo que funcionó mejor de lo esperado

La integración entre módulos fue más fluida de lo que el equipo anticipaba. Definir los contratos de API al inicio del proyecto —antes de que cada miembro arrancara su desarrollo— eliminó la mayoría de los problemas de compatibilidad que suelen aparecer en proyectos distribuidos de este tipo. Cuando P4 terminó el módulo de captura CDP y P2 ya tenía el endpoint receptor implementado, la integración se completó sin fricción.

El uso de variables de entorno para controlar el modo de operación del módulo de IA resultó ser una decisión especialmente acertada. El modo mock (`MOCK_PLAYWRIGHT=true`) permitió desarrollar y validar el ciclo completo de análisis de forma independiente, sin depender de que P2 y P3 tuvieran sus módulos listos. Esto desbloqueó el desarrollo en paralelo real y redujo los tiempos de espera entre módulos.

La elección de Claude como motor de análisis superó las expectativas iniciales en términos de precisión. La detección de inyecciones SQL en DVWA alcanzó niveles de confianza superiores al 90% con prompts relativamente sencillos, lo que confirma que la calidad del prompt es más determinante que la complejidad del código de análisis.

Lo que fue más difícil de lo previsto

La coordinación entre cinco módulos con dependencias cruzadas generó cuellos de botella que no estaban completamente previstos en el diseño inicial. El módulo de backend actúa como nexo central del sistema, lo que convirtió a P2 en el punto de mayor presión del proyecto: cualquier endpoint pendiente en el backend bloqueaba simultáneamente a varios módulos. En retrospectiva, haber paralelizado más agresivamente el desarrollo del backend desde el inicio habría reducido estos bloqueos.

La gestión del entorno Python en Windows presentó más fricción de la esperada. La incompatibilidad entre la versión inicial de la librería `anthropic` y Python 3.14, la exposición accidental de una API key en el repositorio o la configuración del entorno virtual en distintos sistemas operativos fueron incidencias menores que, sumadas, consumieron tiempo de desarrollo que no estaba previsto en el plan inicial.

Qué haríamos diferente si empezáramos de nuevo

Estableceríamos la convención de commits desde el primer commit del repositorio, no como corrección posterior. La convención de commits es un estándar de calidad que penaliza

directamente la nota cuando no se aplica, y su adopción tardía en un historial ya creado es costosa de corregir sin reescribir el historial.

Definiríamos un entorno de integración compartido desde el inicio. Durante el desarrollo, cada módulo se probó de forma aislada contra DVWA en local. Un entorno de integración compartido en Hetzner desde la primera semana habría permitido detectar antes los problemas de integración real entre módulos y habría dado más tiempo para resolverlos.

| Aprendizajes sobre integración de sistemas complejos

Este proyecto confirma que la dificultad de un sistema distribuido no está en la complejidad de cada módulo individual, sino en la gestión de sus interfaces. Un módulo técnicamente excelente que no cumple el contrato de API acordado bloquea a todos los módulos que dependen de él. La disciplina en la definición y el respeto de los contratos de integración es tan importante como la calidad del código.

La inteligencia artificial como componente de un sistema mayor introduce un tipo de incertidumbre diferente al del código determinista. Los tiempos de respuesta variables, los costes por llamada y la naturaleza probabilística de las clasificaciones requieren diseñar el sistema de forma que pueda operar de forma degradada cuando la IA no está disponible o cuando su respuesta no supera el umbral de confianza requerido.

SECCIÓN 8

Road map de mejora para la Práctica 2

Sección pendiente de entrega por P4 — Límite: 19 de Mayo de 2026